

Seguridad y Privilegios

Alumnos: Lautaro Perez, Federico Huergo, Ciro Feijoo, Marcos Hoyuelos

Universidad Tecnológica Nacional - Regional Mar del Plata

Profesores: Adrián Aroca y Rómulo Currao

Índice:

1.	Introducción.....	3
2.	Usuarios.....	4
2.1.	Definición	
2.2.	Categorías	
2.2.1.	Root o superusuarios	
2.2.2.	Usuarios de aplicación o cuentas de servicios	
2.2.3.	Usuarios e administración/consulta	
2.3.	Creación	
2.4.	Modificar	
2.5.	Eliminar	
3.	Privilegios.....	7
3.1.	Definición	
3.2.	Otorgar privilegios	
3.3.	WITH GRANT OPTION	
3.4.	Ver privilegios	
3.4.1.	Ver privilegios de un usuario	
3.4.2.	Ver privilegios propios	
3.5.	Revocar privilegios	
3.6.	Privilegios más utilizados	
3.6.1.	Privilegios de datos (DML)	
3.6.2.	Privilegios de estructura (DDL)	
3.6.3.	Privilegios administrativos	
3.7.	Orden de evaluación de privilegios. Jerarquía de capas.	
3.8.	Ejemplo práctico: Acceso denegado a un usuario	
3.9.	Ejemplo práctico: Concurrencia de dos usuarios con uso de transacciones.	
4.	Roles.....	12
4.1.	Definición	
4.2.	Crear roles	
4.3.	Otorgar/Revocar privilegios	
4.4.	Eliminar privilegios de un rol	
4.5.	Otorgar un rol a un usuario	
4.6.	Activación y establecer por defecto	
4.7.	Establecer un rol por defecto para un usuario	
4.8.	WITH ADMIN OPTION	
4.9.	Ver los privilegios de un rol	
4.10.	Revocar un rol de un usuario	
4.11.	Eliminar un rol	
4.12.	Permitir a un usuario crear roles	
4.13.	Equivalencias entre usuarios y roles	
4.14.	Auditor de herencias	
4.15.	Aclaraciones	
5.	Conclusiones.....	16
5.1.	Aclaraciones	
5.2.	Conclusiones finales	
6.	Referencias.....	17

1. Introducción

Las brechas de seguridad pueden resultar en pérdidas de datos, acceso autorizado a información sensible y manipulación de registros por parte de un atacante. Es por esto que la seguridad en bases de datos es de **necesidad crítica** para proteger la confidencialidad, integridad y disponibilidad de la información.

Sin las medidas necesarias para asegurar la información de una base de datos, una organización puede enfrentar grandes riesgos:

- **Consecuencias legales y financieras:** Debido a una posible filtración de datos, puede enfrentar sanciones costosas.
- **Daño reputacional:** Los usuarios pierden confianza.
- **Interrupción de servicios:** Ataques que pueden causar la caída del sistema o la corrupción de datos esenciales.

Uno de los aspectos clave de la seguridad es el control de acceso y autenticación. Esto implica el uso de cuentas de usuario con contraseñas seguras y la asignación de privilegios específicos.

En MySQL los privilegios son el **derecho o permiso** que se le otorga a un usuario para ejecutar una sentencia SQL específica o acceder a un objeto determinado del sistema. Son la base de la seguridad informática de nuestro sistema.

A la hora de otorgar privilegios es importante seguir el Principio de Menor Privilegio (o POLP); *"Cualquier usuario o proceso debe tener únicamente los privilegios mínimos necesarios para realizar su función"*. De esta manera podemos garantizar que si un usuario se ve comprometido o comete algún error, la base de datos sufrirá el menor daño posible.

2. Usuarios

2.1 ¿Cómo definimos a un usuario en MySQL?

Los usuarios se identifican por su nombre de usuario y por la dirección de IP desde la cual están accediendo a la base de datos. El formato para escribir esta identificación es:

```
'user'@'host'
```

Donde 'User' es el nombre de usuario y 'Host' es la dirección IP o el servidor desde el cual el usuario está accediendo a la base de datos.

2.2 Categorías de usuarios:

En un entorno profesional, las cuentas de acceso no se crean de forma genérica, sino que se clasifican en categorías estrictas según su función, alcance y nivel de confianza dentro de la infraestructura.

2.2.1 Root o Superusuario:

Es la cuenta de control total preinstalada en el motor de base de datos. Posee el máximo nivel de privilegios globales; puede borrar y crear bases de datos enteras, alterar la configuración del servidor y gestionar el almacenamiento físico.

A diferencia de otros usuarios, el usuario root se crea con el servidor y su uso se limitará a emergencias y configuraciones iniciales (no se usa root para el acceso del día a día). En su lugar se deberían crear otros usuarios especializados.

El acceso remoto al usuario root deberá ser deshabilitado, permitiendo únicamente conexiones dentro del entorno local.

2.2.2 Usuarios de aplicación o cuentas de servicio:

Son credenciales creadas exclusivamente para que el software (servidores web, APIs, microservicios o aplicaciones móviles) interactúe de forma automatizada con los datos.

Este tipo de usuarios no corresponden a una persona real, su acceso a las tablas específicas del sistema está limitado y su origen se restringe a la dirección IP exacta en que corre el servidor de la aplicación.

Jamás deben poseer permisos estructurales de tipo DDL (como **DROP** o **ALTER**).

2.2.3 Usuarios de Administración/Consulta:

Son las cuentas asignadas a personas reales del equipo técnico (desarrolladores, analistas de datos, auditores o administradores de infraestructura).

Cada cuenta es individual por persona, así permite la correcta auditoría de los logs (acciones/registros del usuario dentro de la base de datos).

Los privilegios de este tipo de cuentas suelen gestionarse de forma masiva a través de los Roles.

2.3 ¿Cómo se crea un usuario?

```
CREATE USER 'estudiante'@'localhost' IDENTIFIED BY 'Password123!';
```

Crear usuario limitando sus consultas por hora para evitar abusos:

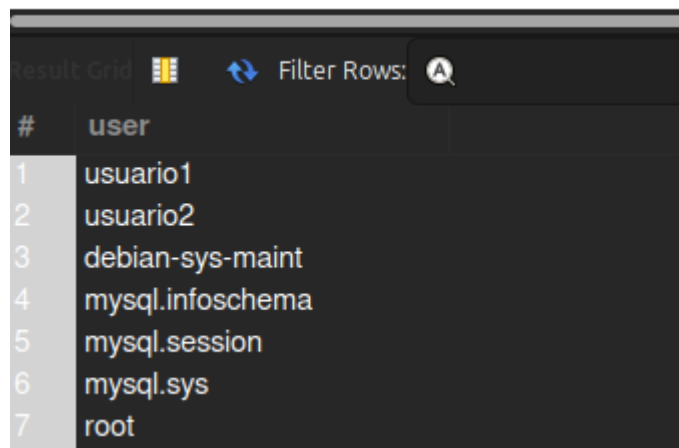
```
CREATE USER 'estudiante'@'localhost' IDENTIFIED BY 'Password123!'
WITH MAX_QUERIES_PER_HOUR 100
     MAX_CONNECTIONS_PER_HOUR 5;
```

Con la cláusula `IDENTIFIED BY` se define la contraseña con la que ese usuario deberá acceder. Los usuarios se guardan en una de las bases de datos del sistema llamada “mysql.user”, por lo tanto si queremos ver una lista de los usuarios en nuestro sistema deberemos utilizar la siguiente query:

```
SELECT user, host FROM mysql.user
```

Ejemplo del resultado de un Query a mysql.user:

81 • `SELECT user FROM mysql.user;`



#	user
1	usuario1
2	usuario2
3	debian-sys-maint
4	mysql.infoschema
5	mysql.session
6	mysql.sys
7	root

Se muestran dos usuarios creados, el usuario root y otros usuarios propios del sistema.

2.4 ¿Cómo se modifica un usuario?

Para modificar las propiedades de un usuario existente (como cambiar su contraseña, su host de conexión o bloquear la cuenta), se utiliza la sentencia `ALTER USER`.

Cambiar contraseña:

```
ALTER USER 'estudiante'@'localhost' IDENTIFIED BY 'Password123456!';
```

Acá hay un margen de matices que es apropiado aclarar; un usuario cualquiera puede cambiar su propia contraseña (más no su configuración) a pesar de carecer de los privilegios para hacerlo con otros usuarios (específicamente el de `CREATE USER`) de las siguientes formas:

- Con `ALTER USER` (de la misma forma en que se mostró), pero MySQL lo permite únicamente si el usuario se apunta a sí mismo mediante la función `USER()`.

```
ALTER USER USER() IDENTIFIED BY 'NuevaContrasenia123456';
```

- Usando el comando de sesión `SET PASSWORD`, el cual fue creado para la gestión de credenciales autónomas. Si no se especifica a qué usuario aplicar el cambio, el motor asume por defecto que es para el usuario que está ejecutando la sesión actual:

```
SET PASSWORD = 'NuevaContrasenia123456';
```

Sin embargo, si el administrador activó la variable global del sistema `read_only` o `super_read_only` (usado en escenarios avanzados en administración de servidores), el usuario común no podrá cambiar su contraseña.

Bloquear o desbloquear temporalmente una cuenta sin necesidad de eliminarla:

```
ALTER USER 'estudiante'@'localhost' ACCOUNT LOCK;  
ALTER USER 'estudiante'@'localhost' ACCOUNT UNLOCK;
```

Renombrar un usuario:

```
RENAME USER 'estudiante'@'localhost' TO 'alumno'@'localhost';
```

2.5 ¿Cómo se elimina un usuario?

```
DROP USER 'estudiante'@'localhost';
```

3. Privilegios

3.1 ¿Qué es un privilegio?

Un privilegio es un derecho o permiso explícito que se le otorga a un usuario para ejecutar una sentencia SQL específica o acceder a un objeto determinado del sistema (como una base de datos, tabla o columna).

Surgen de la necesidad de controlar el acceso que cada usuario tiene a la base de datos. Si todos los usuarios tuvieran los mismos privilegios que el usuario root, controlar la seguridad de nuestra base de datos sería imposible.

Siendo administradores en una base de datos ya establecida en un entorno profesional, cuando creamos un usuario, este carece de privilegios, por lo que no puede acceder a ninguna tabla de ninguna base de datos de nuestro sistema.

En cambio, en instalaciones limpias de MySQL, por defecto este otorga el privilegio `USAGE` y da acceso implícito a la base de datos `INFORMATION_SCHEMA` (para que el usuario pueda ver sus propios metadatos) y, a veces, a la base de datos `test`.

Existen varios niveles de privilegio que se pueden asignar a un usuario:

- **Global:** Afectan a todas las bases de datos del sistema.
- **Base de Datos:** Afectan a una base de datos en específico.
- **Tabla:** Se concede acceso a unas sola tablas puntualmente.
- **Columna:** Se concede acceso a solo unas columnas específicas de cualquier tabla.
- **Rutinas almacenadas:** Se concede acceso a Procedimientos almacenados o a UDFs.
- **Proxy:** Se conceden a un usuario los mismos privilegios que ya tiene otro.

3.2 Cómo asignar privilegios a un usuario

```
GRANT
    PRIVILEGIO1, PRIVILEGIO2,...
ON
    'MiBaseDeDatos'.'MiTabla'
TO
    'user'@'host';
```

A continuación se muestra la sintaxis para cada uno de los niveles de privilegio:

Global:

```
GRANT
    SELECT, UPDATE
ON
    *.*
TO
    'estudiante'@'localhost';
```

Base de Datos:

```
GRANT
    SELECT, UPDATE
ON
    base_datos.*
TO
    'estudiante'@'localhost';
```

Tabla:

```
GRANT
    SELECT, UPDATE
ON
    base_datos.tabla1
TO
    'estudiante'@'localhost';
```

Columna:

```
GRANT
    SELECT(columna1,columna2),
    UPDATE(columna1)
ON
    base_datos.tabla1
TO
    'estudiante'@'localhost';
```

Rutinas Almacenadas:

```
GRANT
    EXECUTE
ON
    PROCEDURE sp_calcular_dato
TO
    'estudiante'@'localhost';
```

Proxy:

```
GRANT
    PROXY
ON
    'usuario1'@'localhost'
TO
    'estudiante'@'localhost';
```


Si queremos otorgarle a un usuario todos los privilegios sobre una base de datos usaremos **ALL**:

```
GRANT
  ALL
ON
  base_datos.*
TO
  'estudiante'@'localhost';
```

3.3 WITH GRANT OPTION

Si queremos que un usuario tenga la capacidad de otorgar los privilegios que posee debemos agregar la cláusula **WITH GRANT OPTION**:

```
GRANT
  SELECT, INSERT
ON
  base_datos.tabla1
TO
  'estudiante'@'localhost'
WITH GRANT OPTION;
```

El alcance de esta opción siempre será el mismo que tenía el usuario cuando le fue otorgada la opción. Por ejemplo, en el caso de arriba, el usuario estudiante solo podrá otorgar permisos de **SELECT** e **INSERT** sobre la “tabla1” de baseDeDatos.

Además, **WITH GRANT OPTION** también se hereda al otorgar derechos, esto quiere decir que si el usuario A le da permisos al usuario B con **GRANT OPTION**, el usuario B ahora también puede darle esos permisos a un usuario C.

Por otro lado, esta cláusula también otorga la posibilidad de hacer **REVOKE** de los privilegios que se concedieron con el **WITH GRANT OPTION**.

Sin embargo, **WITH GRANT OPTION** posee un riesgo de escalada de privilegios considerable; Si se le da a un usuario permisos de **SELECT** con esta cláusula, este usuario puede crear un rol malicioso, meter ese permiso ahí y transferirlo a cualquiera.

3.4 Cómo ver los privilegios

Si un usuario no tiene los privilegios administrativos para consultar los permisos de otros, puede auditar sus propios accesos vigentes utilizando la función **CURRENT_USER()** u omitiendo la palabra **FOR** en la siguiente cláusula.

3.4.1 Ver privilegios de un usuario.

```
SHOW GRANTS FOR 'estudiante'@'localhost';
```

3.4.2 Ver privilegios propios

```
SHOW GRANTS;
```

3.5 Cómo eliminar los privilegios de un usuario

```
REVOKE
    PRIVILEGIO1, PRIVILEGIO2,...
ON
    NIVEL_PRIVILEGIO
FROM
    'user'@'host';
```

3.6 Privilegios más utilizados:

3.6.1 Privilegios de Datos (DML):

Nombre	Qué hace
“SELECT”	Permite al usuario leer datos de tablas (sentencia SELECT).
“INSERT”	Permite al usuario agregar nuevos registros a las tablas (sentencia INSERT).
“UPDATE”	Permite al usuario modificar registros existentes (sentencia UPDATE).
“DELETE”	Permite al usuario eliminar registros de las tablas (sentencia DELETE).

3.6.2 Privilegios de Estructura (DDL):

Nombre	Qué hace
“CREATE”	Permite al usuario crear nuevas bases de datos y tablas
“DROP”	Permite al usuario eliminar bases de datos y tablas existentes..
“ALTER”	Permite al usuario modificar la estructura de tablas existentes (añadir columnas, cambiar tipos de datos, etc.).
“INDEX”	Permite al usuario crear o eliminar índices en las tablas.

3.6.3 Privilegios Administrativos:

Nombre	Qué hace
“ALL PRIVILEGES:”	Otorga todos los permisos disponibles, excepto GRANT OPTION (ideal solo para el usuario administrador o root).
“CREATE USER”	Permite crear, renombrar o eliminar cuentas de usuario.

3.7 Orden de evaluación de privilegios. Jerarquía de capas.

MySQL no revisa los privilegios al azar cuando un usuario ejecuta una consulta, sigue un orden estrictamente jerárquico de arriba para abajo:

1. **Capa Global** (mysql.user): Revisa si el usuario tiene permisos para todo el servidor (ejemplo *.*). Si lo tiene, le da acceso inmediato.
2. **Capa de Base de Datos** (mysql.db): Si no es global, busca si tiene permisos específicos sobre el esquema específico (ejemplo base_datos.*).
3. **Capa de Tablas** (mysql.tables_priv): Si no, busca el permiso a nivel tabla.
4. **Capa de Columnas** (mysql.columns_priv): Finalmente, evalúa si el permiso está restringido a campos específicos.

En MySQL **los permisos se suman, no se restringen**. Si un usuario tiene **SELECT** a nivel global, es imposible quitárselo en una tabla específica usando **REVOKE**. Para restringir una tabla, el usuario no debe tener permisos globales.

3.8 Ejemplo práctico: Acceso denegado a un usuario

En el siguiente ejemplo se usa la sesión del “usuario1” que en este caso tiene únicamente permisos de lectura y escritura sobre la tabla “empleados”

```
1 SET ROLE ALL;
2 • USE BaseDeDatosEjemplo;
3 • SELECT * FROM empleados;
```

#	id_empleado	nombre	apellido	email	suelo	fecha_ingreso	id_departamento
1	1	Carlos	Gómez	carlos.gomez@empresa.com	450000.00	2024-01-15	1
2	2	Ana	Martínez	ana.martinez@empresa.com	520000.00	2023-06-10	1
3	3	Lucía	López	lucia.lopez@empresa.com	380000.00	2025-02-01	2
4	4	Mariano	Rodríguez	mariano.rod@empresa.com	410000.00	2024-11-20	3
5	5	Sofía	Fernández	sofia.f@empresa.com	600000.00	2022-03-14	NULL

La query sobre la tabla empleados funciona correctamente.

```
1 SET ROLE ALL;
2 • USE BaseDeDatosEjemplo;
3 • SELECT * FROM departamentos;
```

#	Time	Action	Message	Duration / Fetch
1	21:54:12	SELECT * FROM departamentos LIMIT 0, 1000	Error Code: 1142. SELECT command denied to user 'usuario1'@'localhost' for table 'departamentos'	0.00032 sec

La query sobre la tabla “departamentos” falla y el sistema lanza un error advirtiéndole que se carece de los permisos necesarios.

3.9 Ejemplo práctico: Concurrencia de dos usuarios con uso de transacciones.

En el siguiente ejemplo se muestran las queries de dos usuarios que están intentando modificar el mismo elemento de manera concurrente. En el mismo se ve como la transacción del “usuario1” bloquea momentáneamente la transacción del “usuario2”.

Query 1

Limit to 1000 rows

1 • SET ROLE ALL;

2 • USE BaseDeDatosEjemplo;

3

4 -- 1 El usuario 1 inicia una transaccion

5 • START TRANSACTION;

6

7 -- 2 Modifica el sueldo del usuario de id 1

8 • UPDATE empleados SET sueldo = 480000.00 WHERE id_empleado = 1;

9

10 -- 3 [En este momento ejecuto las queries del usuario dos para simular una ejecucion en paralelo]

11

12 -- 6 El usuario 1 termina su transaccion y destraba la columna de sueldo del empleado de id 1

13 • COMMIT;

Action Output

#	Time	Action	Message
✓ 1	22:25:42	SET ROLE ALL	0 row(s) affected
✓ 2	22:25:42	USE BaseDeDatosEjemplo	0 row(s) affected
✓ 3	22:25:42	START TRANSACTION	0 row(s) affected
✓ 4	22:25:42	UPDATE empleados SET sueldo = 480000.00 WHERE id_e...	0 row(s) affected Rows matched: 1 Changed: 0 Warnings: 0

En la primera imagen se ve la transacción que el usuario1 realiza, la misma se deja a medias. En cada imagen está comentado el orden en que cada usuario realiza cada paso, esto se hace en un orden específico para simular la concurrencia de transacciones.

Query 1 ↻				
Limit to 1000 rows				
1	•	SET ROLE ALL;		
2	•	USE BaseDeDatosEjemplo;		
3				
4	--	4 El usuario2 inicia su propia transacción en paralelo		
5	•	START TRANSACTION;		
6				
7	--	5 Intentar modificar al mismo empleado		
8	•	UPDATE empleados SET sueldo = 500000.00 WHERE id_empleado = 1;		
9				
10	--	7 Como el usuario1 ya se termino su transaccion, esta segunda transaccion se destraba sola		
11	•	COMMIT;		

Action Output ▾				
#	Time	Action	Message	Duration / Fetch
✓ 1	22:22:48	SET ROLE ALL	0 row(s) affected	0.00018 sec
✓ 2	22:22:48	USE BaseDeDatosEjemplo	0 row(s) affected	0.00014 sec
✓ 3	22:22:48	START TRANSACTION	0 row(s) affected	0.00016 sec
4	22:22:53	UPDATE empleados SET sueldo = 500000.00 WHERE id_e...	Running...	?

En esta segunda imagen se ve la transacción del usuario2, la cual queda en espera debido a que la transacción del usuario1 aún no ha llegado a su commit.

Una vez que la transacción del usuario1 llega a su commit automáticamente se desbloquea la transacción del usuario2.

✓ 9	22:27:49	UPDATE empleados SET sueldo = 500000.00 WHERE id_e...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	11.576 sec
✓ 10	22:28:00	COMMIT	0 row(s) affected	0.0056 sec

En esta última imagen se puede apreciar que la transacción del usuario2 estuvo en espera por 11 segundos.

4. Roles

4.1 ¿Qué es un Rol?

Los roles en MySQL son conjuntos de privilegios con nombre (ej. lector, admin) que facilitan la gestión de seguridad al asignar permisos a grupos de usuarios en lugar de individualmente. Los roles se introdujeron a MySQL en la versión 8.0.

4.2 Crear un rol:

```
CREATE ROLE nombre_rol;
```

4.3 Asignar privilegios a un rol creado:

```
GRANT
    SELECT,
    INSERT
ON
    base_datos.*
TO
    nombre_rol;
```

4.4 Eliminar privilegios de un rol creado:

```
REVOKE
    SELECT
ON
    base_datos.*
FROM
    nombre_rol;
```

4.5 Asignar un rol a un usuario.

Los roles asignados de esta forma quedan asociados al usuario, pero en el momento que este se conecte dichos roles no estarán activos:

```
GRANT nombre_rol TO 'usuario'@'localhost';
```

La manera que tiene MySQL de distinguir cuando estamos asignando un rol y cuando estamos asignando un privilegio es la existencia de la cláusula **ON**. Nunca asignaremos un rol sobre una base de datos específica.

4.6 Activar un rol para la sesión actual.

¿Por qué los roles no están activos? Sencillo. MySQL introdujo esto por seguridad analítica. En producción un usuario puede tener asignados roles muy potentes, pero es preferible iniciar sesión con un rol básico para evitar ejecutar comandos destructivos por error. Por lo que solo debe activarlo cuando es consciente de que debe ejecutar comandos de mantenimiento.

Esto nos permite activar los roles de nuestro usuario que estaban “dormidos” luego de logearnos. Esta activación de los roles solo persistirá por la sesión actual:

```
SET ROLE nombre_rol;
```

4.7 Establecer un rol por defecto para un usuario.

Así es cómo podemos definir roles por defecto para que se activen siempre que se loguea determinado usuario:

```
SET DEFAULT ROLE nombre_rol TO 'usuario'@'localhost';
```

4.8 WITH ADMIN OPTION.

Este comando cumple un rol equivalente al de usar WITH GRANT OPTION con los privilegios, pero se aplica a los roles:

```
GRANT
    nombre_rol
TO
    'usuario'@'localhost'
WITH ADMIN OPTION;
```

Ahora 'usuario'@'localhost' también podrá otorgar el rol que se le está dando. Esta cláusula también da la posibilidad de revocar Roles.

WITH ADMIN OPTION no otorga la posibilidad de otorgar los privilegios que contiene un rol de forma individual, sólo permite otorgar el rol completo. Para otorgar los privilegios de cada rol por separado, tenemos que asignar cada uno de los privilegios que queremos poder otorgar individualmente con WITH GRANT OPTION a la hora de dárselos al rol correspondiente.

4.9 Ver los privilegios que determinado rol ofrece.

Con este comando el sistema “abre” el rol 'nombre_rol' y muestra los privilegios que este rol en particular engloba (junto con otros privilegios y roles que el usuario podría tener):

```
SHOW GRANTS FOR 'usuario'@'localhost' USING nombre_rol;
```

4.10 Revocar un rol de un usuario:

```
REVOKE nombre_rol FROM 'usuario'@'localhost';
```

4.11 Eliminar un rol.

Al eliminar un rol, este automáticamente se revoca de cualquier usuario que lo tuviera.

```
DROP ROLE nombre_rol;
```

4.12 Cómo dar a un usuario privilegios para crear, revocar y otorgar roles.

Para conseguir esto debemos otorgarle al usuario el privilegio ROLE_ADMIN.

```
GRANT
    ROLE_ADMIN
ON
    *.*
TO
    'usuarioExterno1'@'%';
```

Hay que tener en cuenta que ROLE_ADMIN solo puede asignarse de manera global para el sistema (*.*).

4.13 Equivalencia entre usuarios y roles

Un rol se diferencia de un usuario principalmente en que carece de autenticación por contraseña directa y su campo de host suele ser un comodín interno, sirviendo únicamente como contenedor de privilegios.

Internamente, MySQL trata de manera equivalente a los roles y a los usuarios, es por esto que podemos usar GRANT para asignar todos los privilegios de un usuario a otro usuario:

```
GRANT
    'usuario1'@'localhost'
TO
    'usuario2'@'localhost';
```

De esta forma los privilegios de 'usuario1' serían asignados a 'usuario2'. (Siempre y cuando el usuario que está ejecutando la sentencia tenga la facultad para pasar todos los roles y privilegios que está acción implica)

Así como se le puede asignar un rol a un usuario, también se puede asignar un rol a otro rol, por lo que es posible crear una **Jerarquía de roles** donde los mismos pueden heredar unos de otros:

```
/*
Se crea un rol 'lector_datos' asignando privilegios para hacer SELECT en
```



```
la base de datos 'mi_empresa'
*/

CREATE ROLE lector_datos;

GRANT
SELECT
ON
    mi_empresa.*
TO
    lector_datos;

/*
Se crea el rol 'editor_datos' asignando los privilegios del rol
'lector_datos'
*/

CREATE ROLE editor_datos;
GRANT
    lector_datos
TO
    editor_datos;

/*
Por último se asigna a 'editor_datos' los privilegios necesarios para
editar datos en 'mi_empresa'
*/

GRANT
    INSERT, UPDATE
ON
    mi_empresa.*
TO
    editor_datos;

/*
Ahora cualquier usuario con el rol 'editor_datos' podrá leer y editar la
base de datos 'mi_empresa' */
```

4.14 Auditor de herencias.

```
SELECT ROLES_GRAPH_ML();
```

Esta función devuelve un elemento en formato gráfico (XML) que describe exactamente cómo se conectan los roles asignados y quién hereda de quién.

5. Conclusiones

5.1 Aclaraciones

Antes de concluir, es necesario aclarar unas cuestiones:

Diferencia entre REVOKE y DROP: El primero quita los privilegios, pero el usuario/rol sigue existiendo. El segundo destruye por completo el usuario o rol del sistema.

El uso de FLUSH PRIVILEGES: Es el comando que le indica a MySQL “leer las tablas de configuración de nuevo luego de haber cambiado los accesos”. Aunque con `GRANT` y `REVOKE` modernos no es estrictamente obligatorio, en la administración de servidores se sigue usando por seguridad.

Solo es necesario cuando se modifican las tablas del sistema (`mysql.user`, `mysql.db`) de forma directa mediante sentencias DML tradicionales (`INSERT`, `UPDATE`, `DELETE`). Cuando un administrador utiliza comandos nativos de control de accesos como `GRANT`, `REVOKE`, `CREATE USER` o `ALTER USER`, el motor de MySQL actualiza los privilegios en la memoria de manera inmediata y automática. Por lo tanto, ejecutar `FLUSH PRIVILEGES` después de un `GRANT` moderno es redundante e innecesario.

5.2 Conclusiones finales

La elaboración de este informe nos permite determinar que la seguridad en MySQL no debe ser un añadido posterior al desarrollo, sino un pilar arquitectónico fundamental desde la misma concepción de la base de datos.

La correcta implementación de un modelo basado en roles es la manera más eficaz de proteger nuestra base de datos frente a las amenazas legales y financieras que una brecha de seguridad representa. Además, agrupar privilegios bajo identidades lógicas reduce la redundancia de código.

Asimismo, poder asignar facultades específicas a cada usuario nos permite organizar el entorno y minimizar el error humano. Garantiza que un error operativo o una vulnerabilidad de una cuenta no se traduzca en una falla del sistema, haciendo que cada entidad interactúe con la base de datos bajo un marco de control estricto, mantenible y escalable en el tiempo.

Por otro lado, el control del ciclo de vida del usuario establece un marco de trazabilidad absoluta. En un entorno profesional, esto mitiga los riesgos de sanciones legales, pérdidas financieras y daños reputacionales.

6. Referencias

Seguridad: <https://dev.mysql.com/doc/refman/8.0/en/security.html>

Roles: <https://dev.mysql.com/doc/refman/9.7/en/roles.html>

MySQL Tutorial: <https://www.mysqltutorial.org/mysql-administration/>

Usuario Root y Privilegios:

[https://mysqlja.com.ar/informatica/root-user-mysql/#todo lo que necesitas saber sobre el poderoso_comando_mysql_root](https://mysqlja.com.ar/informatica/root-user-mysql/#todo%20lo%20que%20necesitas%20saber%20sobre%20el%20poderoso%20comando%20mysql_root)

Tipos de privilegios: <https://dev.mysql.com/doc/refman/9.7/en/privileges-provided.html>

Niveles de privilegios: <https://conclase.net/mysql/curso/cap13>

Uso de roles: <https://dev.mysql.com/doc/refman/8.0/en/roles.html>

Acceso a usuarios y privilegios desde MySQL Workbench:

<https://dev.mysql.com/doc/workbench/en/wb-mysql-connections-navigator-management-users-and-privileges.html>

Gestionar usuarios y permisos: <https://youtu.be/8cbSZ7i7w7Q?si=EEoXYEsp80qp2yqw>